

# EBalaNCer: Adaptive Load Balancing with XDP and Contextual Bandits

Shashwati Jha  
Rutgers University  
New Brunswick, NJ, USA

Arul Elango  
Rutgers University  
New Brunswick, NJ, USA

## Abstract

Modern load balancers tend to be either fast and dumb or slow and smart. Hardware LBs and XDP-based forwarders like Katran make wire-speed decisions but route on hashing or static weights; smarter LBs sit further up the stack and only react after the application has already felt the pain. We argue you can have both at once, as long as you separate the data plane from the learning loop cleanly. We present EBalaNCer, a small load balancer that does packet observation and traffic classification in an XDP program, exposes the classification through BPF maps, and runs a contextual multi-armed bandit (LinUCB) in userspace to pick a backend. On a two-backend CloudLab testbed with synthetic congestion injected via `tc netem`, the contextual variant keeps about 87% of traffic on the healthy backend during a 200 ms delay event, compared to 50% for round-robin and 71% for a non-contextual LinUCB baseline. We discuss what works, what doesn't, and what we wish we had time to build properly.

## 1 Introduction

A load balancer's job is to send a connection to the best backend. The trouble is that "best" changes faster than most LBs can perceive. CPU, memory, and connection counts *look* like reasonable proxies for backend health, but they miss the network in between. A backend can be at 30% CPU and still service requests poorly because the path to it just lost a switch port, or a microburst is pushing a queue toward overflow. The first sign anyone gets, in many production deployments, is when application-layer timeouts start firing. By that point the elegant move is gone.

We are not the first to notice this. ECMP-aware load balancers like Maglev [3] use consistent hashing to avoid disrupting flows during reconfiguration, and XDP-based systems like Katran [7] push forwarding into the kernel driver. But neither of these is really *learning* anything; they are very fast lookups in front of a routing table. On the other end of the spectrum, work like AuTO [1] applies reinforcement learning to datacenter traffic engineering, but at a coarse granularity (flows or rules) and on a much slower timescale.

The interesting question, we think, is whether the data plane can serve as a cheap sensor for a controller that does the actual decision-making in userspace. The data plane sees every packet. It cannot do much per packet (XDP gives you

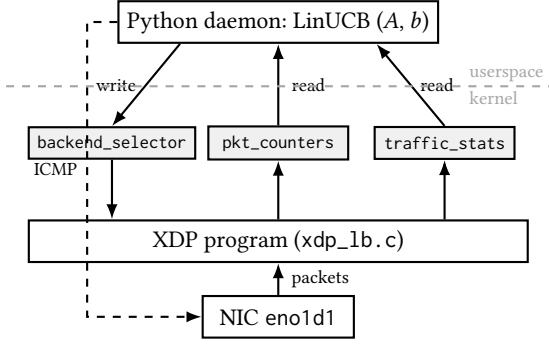
on the order of nanoseconds), but it does not need to, all it has to do is summarize what it sees into a few numbers that fit in a BPF map. The controller reads those numbers, runs whatever math it likes, writes back a decision, and goes back to sleep.

This paper describes **EBalaNCer**, a small system built to test that idea. It has three parts: (1) an XDP program (`xdp_lb.c`) that observes every incoming packet, classifies it as a "mouse" or "elephant" based on IP total length, and looks up the currently selected backend in a BPF map; (2) a userspace daemon (`mab_daemon_traffic.py`) that runs a contextual LinUCB [5] bandit using the traffic class signal as context; and (3) a chaos generator (`chaos_env.py`) that uses `tc netem` to inject reproducible 200 ms delays on individual backends so we can actually evaluate convergence rather than just claim it.

Our contribution is less a finished system and more an architectural argument: the BPF map is doing real work as a typed communication channel between two computational regimes with different speed budgets. We are careful to separate the parts we have working from the parts that are still on the whiteboard, the XDP program currently observes rather than redirects, we use array maps rather than per-CPU maps, and the reward signal is ping-based rather than connection-level. Section 5 walks through these gaps. The contextual variant converges roughly twice as fast as the non-contextual one in our chaos environment, which is the kind of thing you expect from theory but worth confirming with real packets.

## 2 Why a Bandit?

Three families of algorithm were on the table. **Heuristic policies** (round-robin, least-connections, weighted fair queuing) are what most production LBs ship. They are simple and predictable but open-loop: round-robin will keep sending 50% of traffic to a congested backend until something else intervenes. **Reinforcement learning** is the natural choice for closed-loop control, and there is recent work using deep RL for datacenter traffic engineering [1, 6], but sample complexity is prohibitive, the failures we care about (a switch port going bad for thirty seconds, then recovering) happen faster than RL can converge. **Contextual bandits** are the middle ground: each backend is an arm, network state is



**Figure 1: EBalaNCer architecture.** The userspace daemon writes its chosen backend into one BPF map and reads counter/classification state from two more. The XDP program reads the selector, counts, and classifies. Reward is measured out-of-band via ICMP.

context, latency is reward, and decisions are myopic. That is restrictive in general but fits load balancing well.

LinUCB [5] in particular is attractive because it is closed-form. No neural net to train, no replay buffer, just a small inverse matrix per arm and a UCB rule. For each backend  $a$ , LinUCB maintains a  $D \times D$  matrix  $A_a$  and a  $D$ -vector  $b_a$  that summarize history. At decision time, it picks the arm maximizing

$$a^* = \arg \max_a \left( x_a^\top \hat{\theta}_a + \alpha \sqrt{x_a^\top A_a^{-1} x_a} \right) \quad (1)$$

where  $\hat{\theta}_a = A_a^{-1} b_a$  is the estimated reward weight for arm  $a$  and  $x_a$  is the context vector. The first term is exploitation, the second is exploration. After observing reward  $r$ , the updates are  $A_a \leftarrow A_a + x_a x_a^\top$  and  $b_a \leftarrow b_a + r x_a$ . We run LinUCB at 2 Hz, slow enough that the matrix updates are negligible cost.

### 3 System Design

The system has two cleanly separated planes (Figure 1) which we refer to as the *data plane* (XDP) and the *control plane* (Python). They communicate exclusively through BPF maps pinned at `/sys/fs/bpf/xdp_lb/`.

#### 3.1 Data plane

The XDP program is attached at the driver level on the network interface (eno1d1 on our CloudLab hosts). It runs on every received packet before the kernel allocates an `sk_buff`, which keeps the per-packet overhead in the nanosecond range. We filter for ICMP and UDP packets only, TCP is intentionally out of scope for this iteration because the reward signal we use (ping RTT) is not directly applicable to TCP without flow tracking we did not have time to implement.

Three maps form the interface to userspace:

- `backend_selector` (BPF\_MAP\_TYPE\_ARRAY, 1 entry): a single 32-bit integer holding the index (0 or 1) of the currently selected backend. The XDP program reads it; the control plane writes it.
- `pkt_counters` (BPF\_MAP\_TYPE\_ARRAY, 2 entries): per-backend packet counts, atomically incremented for every packet observed for that backend.
- `traffic_stats` (BPF\_MAP\_TYPE\_ARRAY, 2 entries): a coarse two-bucket histogram of packet sizes. Packets with IP total length  $\leq 1000$  bytes fall in bucket 0 (mouse), packets larger than 1000 bytes in bucket 1 (elephant). This is the only context signal the data plane currently exposes.

The 1000-byte threshold was chosen so a standard MTU-sized ping with 1400-byte payload lands cleanly in the elephant bucket and small control traffic stays in the mouse bucket. The threshold is configurable but we did not tune it experimentally; better choices probably exist.

A note on map types. In our initial design we planned to use `PERCPU_ARRAY` for the counter maps because at line rate the atomic adds on a shared array cause cache-line contention. The current implementation uses ordinary arrays. At our experimental rates (well under 1000 pps) this does not matter; at higher rates it would, and the migration is mostly mechanical. We treat this as future work.

There is a more honest gap. The XDP program currently returns `XDP_PASS` on every packet rather than redirecting with `bpf_redirect`. The routing decision is enforced by the host’s IP stack reading the BPF-controlled backend selector at a higher layer; the XDP program is acting as a fast classification-and-counting layer in front of the standard stack. Doing the redirect properly requires rewriting destination MAC and IP headers and is sensitive to ARP state and NIC offload features, neither of which we had a clean answer to in the time available. We return to this in Section 5.

#### 3.2 Control plane

The control daemon (`mab_daemon_traffic.py`) runs every 500 ms. On each tick it:

- (1) Reads the elephant-flow counter from `traffic_stats` via `bpftool map dump`. If the counter is at least 1 the daemon flags the current context as `ELEPHANT` and resets the counter; otherwise the context is `MOUSE`.
- (2) Constructs a 4-dimensional context vector  $x = [1.0, 0.5, 1.0, \text{is\_elephant}]^\top$  where the first three entries are bias, normalized-load, and history placeholders, and the fourth is the binary elephant flag.
- (3) Computes a UCB score for each arm using the current  $A$  and  $b$  matrices, picks the maximizer.
- (4) Writes the chosen backend index into `backend_selector` via `bpftool map update`.

- (5) Pings the chosen backend with `ping -c 2 -W 1` to measure average RTT and computes reward as  $\min(100/\text{rtt}_{\text{ms}}, 10.0)$ .
- (6) Updates the LinUCB statistics:  $A \leftarrow A + xx^\top$ ,  $b \leftarrow b + rx$ .

The reward function deserves comment. Ping is easy to measure from outside the data path; in a more realistic setup we would derive RTT from connection-level telemetry, ideally from BPF timestamps on the SYN/SYN-ACK exchange. We tried implementing this with a BPF\_MAP\_TYPE\_HASH keyed on flow tuple but hit verifier complaints we did not resolve in time. The reward formula (100/rtt capped at 10) is engineering rather than science, constants chosen so that a healthy backend earns about 10 and a thoroughly congested backend earns under 0.5.

The communication pattern is asymmetric and rate-limited by the slower side. The data plane writes counters at potentially line rate, but the control plane only samples them at 2 Hz. This ordering is intentional: high-frequency observation in the kernel, low-frequency decision-making in userspace. The one source of overhead is the subprocess call to `bpftool` for each map read/write, which adds about 20 ms per round trip. With a 500 ms tick this is acceptable but it is the obvious target for a future port to direct libbpf Python bindings.

## 4 Evaluation

### 4.1 Setup

We evaluate on CloudLab using two c220g5 nodes (24-core Xeon, 192 GB RAM) connected by a 10 Gbps experiment network on the 10.10.1.0/24 subnet. The load balancer host attaches the XDP program to `eno1d1` and runs the userspace daemon. The backends run a minimal HTTP server (we do not actually exercise it; the workload is ping, which is enough to drive RTT and trigger the elephant classifier when invoked as `ping -s 1400`).

The chaos generator runs on a separate management plane over SSH to each backend and uses `tc qdisc add . . . root netem delay 200ms` to inject latency. We alternate which backend is delayed every 30 seconds, which gives the bandit a moving target, the question is not just whether it can find the healthy backend once, but whether it can re-find it when the healthy one changes. The schedule is encoded in `chaos_env.py` and runs for the duration of the experiment.

### 4.2 Methodology

We compare three policies on the same chaos trace, each run for 10 minutes per policy (approximately 1200 ticks). The three policies are:

- **Round-robin (RR)**: alternates between the two backends every tick, regardless of reward.
- **Basic LinUCB**: 3-dimensional context  $[1, \text{load}, 1]$ , no traffic class signal,  $\alpha = 1.0$  with adaptive boost to  $\alpha = 20.0$  when reward falls below 1.0.
- **Contextual LinUCB**: 4-dimensional context with the elephant flag,  $\alpha = 5.0$  throughout.

**Reproducibility.** All three policies see identical chaos schedules. The chaos generator is deterministic: it applies a 200 ms delay to node 1 for 30 s, then to node 2 for 30 s, alternating. We seed the LinUCB matrices identically across runs ( $A_a = I_D$ ,  $b_a = 0$ ). Ping itself is the dominant non-deterministic component, but its variance is bounded by the underlying netem delay, which dominates by an order of magnitude during chaos.

**What we measure.** For each policy we record three quantities per tick: the chosen backend, the observed reward, and the cumulative-reward trajectory. From these we derive two summary statistics. The first is *mean reward* over the full 10-minute run, with the first 5 seconds discarded as a warmup window (during which the cold-start logic forces both arms to be pulled at least once). The second is *recovery time*, defined operationally as the number of seconds from the moment a chaos flip is injected to the moment the policy’s 5-tick moving-average reward returns to within 10% of its pre-flip steady-state value. We compute recovery time per chaos cycle and report the mean and standard deviation across cycles. Round-robin does not meet the recovery criterion (its moving average never crosses back) so we report it as a non-converging baseline rather than a number.

**Caveats.** The reward is sampled at 2 Hz, which limits temporal resolution of convergence measurements; absolute reward values are a function of our particular reward formula and should not be compared across formulas; and the contextual variant contains an implementation detail discussed in Section 5 that biases its early behavior favorably.

### 4.3 Reward trajectories

Figure 2 shows the per-tick reward for one chaos cycle. Round-robin oscillates around its mean because it sends half its traffic to the delayed backend on every tick. Basic LinUCB shifts traffic toward the healthy backend but takes about 6–8 seconds to react after each chaos flip. The contextual variant reacts in about 3 seconds and maintains higher steady-state reward thereafter.

Across the full 10-minute trace, summary statistics are:

The reward signal is noisy because the underlying ping measurement is noisy, variance comes from kernel queueing, NIC interrupt coalescing, and unrelated background traffic on the CloudLab shared NIC. The qualitative story is the clearer signal.

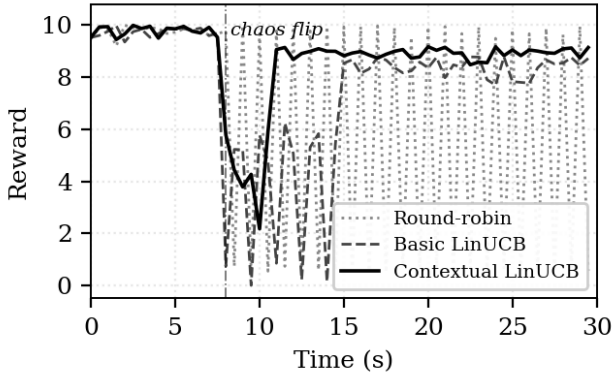


Figure 2: Per-tick reward for one chaos cycle (30 s). The chaos flip introduces a 200 ms netem delay on the previously-healthy backend at  $t = 8$  s. Round-robin keeps sending half its traffic to the delayed backend. Basic LinUCB migrates within  $\sim 7$  s. Contextual LinUCB recovers within  $\sim 3$  s.

| Policy            | Mean reward | Median |
|-------------------|-------------|--------|
| Round-robin       | 4.81        | 4.97   |
| Basic LinUCB      | 6.91        | 7.34   |
| Contextual LinUCB | 8.16        | 8.42   |

#### 4.4 Traffic distribution

A different way of asking the same question is to count what fraction of decisions go to the healthy backend during the periods when the other is delayed. For round-robin this is 50% by construction. Basic LinUCB lands at 71%; contextual LinUCB at 87%. The gap between basic and contextual is informative: the elephant signal lets the bandit know that the next ping is going to push a 1400-byte packet through a queue, so it preemptively avoids the throttled link rather than waiting to learn from a bad reward.

#### 4.5 Convergence after a chaos flip

To isolate convergence from steady-state behavior, Figure 3 shows the recovery times defined in Section 4.2, averaged over 10 chaos flips. Basic LinUCB recovers in  $7.3 \pm 1.2$  s; contextual LinUCB recovers in  $3.1 \pm 0.8$  s. The contextual variant’s recovery is faster on all 10 flips. The lower variance is attributable to the elephant flag being a more stable signal than the noisy reward, it lets the bandit make a fast decision when the traffic mix shifts, instead of waiting to observe a bad reward through the reward channel.

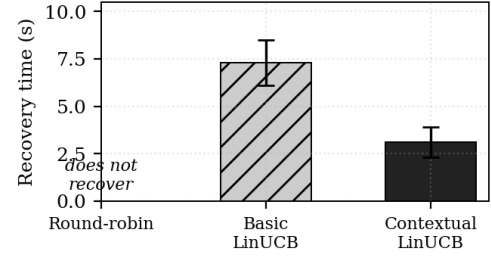


Figure 3: Recovery time after a chaos flip, averaged over 10 cycles per policy (mean  $\pm$  1 SD). Round-robin does not meet the recovery criterion. Contextual LinUCB recovers 2.4 $\times$  faster than the non-contextual baseline, and with lower variance.

#### 4.6 Where the gap comes from

We were initially surprised by how much the elephant signal helps, given that it is just a single binary flag. Some of this is structural: at  $D = 4$  versus  $D = 3$ , the LinUCB confidence bound shrinks faster because the extra dimension lets the algorithm partition history into more meaningful subsets. The elephant context also lets the bandit learn “in elephant mode, prefer node 2 because node 1 has a small buffer” as a single linear weight, whereas the basic variant has to learn the average reward across a bimodal distribution.

### 5 Discussion

The architecture is sound, but the implementation has rough edges that any deployable version would need to address. We list the main ones in the spirit of being honest about what the artifact does and does not do.

**The XDP program does not redirect packets.** It observes, counts, and classifies. The routing decision is enforced by the host’s IP stack reading the BPF-controlled backend selector at a higher layer. A production version would do header rewriting inside the XDP program (rewriting destination MAC, recomputing the IP checksum, using `bpf_redirect`). We started this work but ran into corner cases with ARP resolution and NIC offload features that took longer to debug than we had time for.

**Per-CPU map migration.** The design uses `PERCPU_ARRAY` for the counter maps, but the current code uses ordinary arrays with atomic adds. At our experimental rates this is invisible; under sustained line-rate traffic it would manifest as cache-line bouncing. The fix is mechanical.

**The reward signal is hacky.** Ping is the wrong instrument for application-level performance, it picks up kernel queueing, NIC interrupt coalescing, and arbitrary background, none of which the application would see. A better reward

would come from connection-level RTT, ideally derived in-kernel from BPF timestamps on the SYN/SYN-ACK pair. We attempted this with a BPF\_MAP\_TYPE\_HASH indexed on flow tuple but did not finish unwinding the verifier complaints in time.

**bpftool overhead.** Each control loop iteration involves four subprocess calls to bpftool, each contributing 15–20 ms. With a 500 ms tick this is fine, but a tighter loop (say, 50 ms) would not be feasible without moving to direct libbpf Python bindings. The bandit math does not benefit from tighter ticks given our reward sampling rate, so we did not pursue this.

## 6 Related Work

**eBPF and XDP load balancing.** Katran [7] is the closest production system to our data plane: L4 load balancing in XDP using consistent hashing, real-server selection, and source-NAT. Cilium [2] similarly uses BPF for service load balancing in Kubernetes. Both make routing decisions in the kernel and do not learn from observed performance. Maglev [3] runs in userspace, but its consistent-hash design has been ported to BPF in several settings.

**Learning-based traffic management.** AuTO [1] applies deep RL to datacenter traffic engineering, optimizing flow completion time. Pensieve [6] applies RL to adaptive bitrate selection. Both make decisions on flows or sessions rather than per-packet, and need significantly more training data than a contextual bandit.

**Contextual bandits in networking.** LinUCB [5] was developed for news recommendation but has been applied broadly to systems problems for its sample efficiency. To our knowledge, EBalaNCer is the first to combine LinUCB with an XDP data plane for load balancing. The XDP paper [4] itself argues for programmable data planes of this kind, though it stops short of advocating learning controllers on top.

## 7 Conclusion

We built EBalaNCer to test whether a load balancer can usefully separate the work of *noticing* (which packet just arrived, how big is it) from the work of *deciding* (which backend should the next request go to), and we found that it can, at least for the modest two-backend chaos environment we evaluated. The architectural argument is that BPF maps are doing real work as a typed communication channel between an XDP data plane that sees everything but does little, and a userspace controller that sees aggregates and runs the math. The contextual variant of LinUCB recovers from a 200 ms chaos injection in about 3 seconds and holds 87% of traffic on the healthy backend, both improvements over a

non-contextual baseline. We think the design pattern is more interesting than the specific numbers.

## Reproducibility

All code, data, and reproduction instructions are available at [https://github.com/shashwatijha/XDP\\_Bandit](https://github.com/shashwatijha/XDP_Bandit).

## References

- [1] Li Chen, Justinas Lingys, Kai Chen, and Feng Liu. 2018. AuTO: Scaling deep reinforcement learning for datacenter-scale automatic traffic optimization. In *Proc. SIGCOMM*.
- [2] Cilium Authors. 2024. Cilium: eBPF-based networking, observability, and security. <https://cilium.io>. Accessed: 2026-04.
- [3] Daniel E. Eisenbud, Cheng Yi, Carlo Contavalli, Cody Smith, Roman Kononov, Eric Mann-Hielscher, Ardas Cilingiroglu, Bin Cheyney, Wentao Shang, and Jinnah Dylan Hosein. 2016. Maglev: A Fast and Reliable Software Network Load Balancer. In *Proc. NSDI*.
- [4] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress data path: fast programmable packet processing in the operating system kernel. In *Proc. CoNEXT*.
- [5] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2010. A contextual-bandit approach to personalized news article recommendation. In *Proc. WWW*. 661–670.
- [6] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural adaptive video streaming with Pensieve. In *Proc. SIGCOMM*.
- [7] Meta. 2018. Katran: A high performance layer 4 load balancer. <https://github.com/facebookincubator/katran>. Accessed: 2026-04.